Higher Technological Institute
Engineering Departments
Electrical Department

Digital Clock Using ATmega32A

Team Members:

Abdullah Abdullwahab	*20223355*	***Muhammad Ahmed***	*20220754*
El-Hassan Mohamed	*20220269*	***Saif Eleslam Abdullah***	*20233193*
Mahmoud Naser	*20220390*	***Yousef Mohamed***	*20220246*

Submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Engineering (B.Sc. Engineering) in the course **Microprocessor**

Supervised by:

Dr. Mohamed A. Torad

Associate Professor, Electrical *Department*

10th of Ramadan, engineering department

Higher Technological Institute

November 2025

Al-Sharqia,
Egypt.

`

Abstract

This report presents a compact embedded digital clock and calendar system based on the ATmega32A microcontroller operating at 1 $\boldsymbol{MHz}$. The design demonstrates key embedded systems principles through implementation of a fully functional real-time clock, calendar, and user interface with minimal hardware resources.

A software-based RTC using Timer0 in CTC mode generates precise 10 ms interrupts that drive accurate timekeeping with less than 1% CPU usage and an accuracy of ±2 s/day. A complete Gregorian calendar algorithm handles leap years, month-specific day validation, and automatic date rollover from 2000–2099.

The human–machine interface integrates a 16×2 LCD, 4×4 keypad, three LEDs, and a buzzer, supporting time/date display modes, user input validation, and 12/24-hour format selection. Additional feedback features include LED status indicators, hourly markers, and audible confirmations.

Two software implementations—C and AVR Assembly—are provided, demonstrating trade-offs between readability and optimization. The system occupies only 3–5 KB Flash, 60 B SRAM, and executes each interrupt in under 100 μs, ensuring efficient performance across the ATmega32A's operational range.

Designed for educational and low-cost timekeeping applications, this project highlights interrupt-driven design, timer configuration, calendar logic, and HMI integration. Its modular structure enables extensions such as alarms, temperature sensing, and wireless synchronization, making it an effective platform for embedded systems learning and experimentation.

Table of Contents

List Of Tables

List Of Figures

Chapter 1: Introduction

Accurate timekeeping is essential in many embedded systems, from household devices to industrial controllers. However, implementing a reliable clock on low-cost microcontrollers poses challenges in precision, power use, and cost. While commercial RTC modules like the DS1307 or DS3231 offer high accuracy, they add expense, board space, and complexity. For non-critical uses—such as desk clocks or timers—an internal software-based clock with moderate accuracy (±5 s/day) provides a simpler, cost-effective alternative without external hardware. [1]

1.1 Project Overview

This project addresses the challenges by designing and implementing a fully featured, software-based digital clock system that eliminates the dependency on external RTC hardware. The core objective is to develop a robust, accurate, and user-friendly timekeeping solution using only the internal resources of an ATmega32A microcontroller.

The system demonstrates fundamental embedded programming concepts, including interrupt-driven timing, efficient calendar algorithms, and a state-machine-based user interface. It is engineered to provide practical accuracy for non-critical applications while serving as an excellent educational platform for understanding the intricacies of embedded systems design.

Key Value Propositions:

- **Cost-Effective:** Eliminates the need for external RTC components, reducing the Bill of Materials (BOM).
- **Educational:** Provides transparency into the algorithms and hardware interactions that are hidden within commercial RTCs.
- **Resource-Optimized:** Meticulously designed to operate within the severe constraints of the ATmega32A (2KB SRAM, 32KB Flash). [2]
- **Full Featured:** Implements a comprehensive set of features including time/date setting, multiple display formats, and intuitive user feedback.

1.2 Problem Statement

Accurate timekeeping is a critical requirement across embedded applications ranging from household devices to industrial automation systems. Although conceptually simple, implementing a reliable clock within resource-constrained microcontrollers presents notable challenges involving precision, resource efficiency, and cost. Embedded designers must balance **accuracy, power, complexity, and hardware cost**—often without the aid of external real-time clock (RTC) hardware.

Commercial RTC ICs (e.g., **DS1307**, **DS3231**, **PCF8523**) provide high accuracy and battery backup but increase system cost, PCB space, and design complexity. At scale, even modest component costs (e.g., $5 per unit) significantly impact production budgets. Moreover, these modules act as **black boxes**, limiting educational understanding of timer configuration, interrupt handling, and calendar arithmetic. For many applications—such as clocks or process controllers—moderate accuracy (±5 s/day) is acceptable, making external RTCs unnecessary.

Identified Technical Challenges

1. Hardware Resource Constraints

The **ATmega32A** offers limited resources: **2 KB SRAM**, **32 KB Flash**, an **8-bit architecture**, and a **1 MHz clock**. These constraints demand efficient algorithms, optimized memory usage, and careful interrupt management to ensure reliable operation without compromising performance. [2]

2. Timekeeping Accuracy

Without dedicated RTC hardware, maintaining accuracy depends on the stability of the system clock. Factors such as **crystal tolerance (±20–50 ppm)**, **temperature drift**, **aging**, **load capacitance**, and **interrupt jitter** introduce cumulative errors. Software calibration and correction mechanisms are required to minimize drift and maintain acceptable precision. [2]

3. Calendar Algorithm Complexity

A complete **Gregorian calendar** must handle variable month lengths, leap year rules, and boundary conditions (e.g., February 29 validation, year rollover). Efficient date arithmetic and input validation are essential to avoid logic errors during midnight transitions or user adjustments.

4. User Interface Design

Limited I/O resources—**a 4×4 keypad** [3] and **16×2 LCD** [3]—restrict display space and input flexibility. The interface must manage numeric entry, error prevention, and confirmation feedback within these limitations. Effective **state machine design** and **context-aware prompts** are crucial for usability.

5. Keypad Debouncing and Scanning

Mechanical switches exhibit **5–10 ms contact bounce**, requiring reliable debouncing algorithms. Efficient matrix scanning must prevent **false triggers**, **ghosting**, and **input lag** (<50 ms response), ensuring responsive and accurate user interaction.

6. Interrupt Service Routine (ISR) Efficiency

Timer-driven ISRs must be completed within each **10 ms interrupt period**. Excessive ISR time risks interruption loss or system slowdown. Efficient coding, minimal stack usage, and deferred processing via flag-based signaling maintain timing integrity with minimal overhead. [3]

7. Power-On Initialization

Proper startup sequencing ensures predictable operation. LCD controllers (e.g., HD44780) require strict timing delays; peripherals must initialize in correct order, and all subsystems must default to safe states before normal execution begins. [4]

8. Code Portability and Scalability

Different applications may require distinct formats (12h/24h), display configurations, or hardware variants. A **modular, hardware-abstraction-based architecture** improves portability across LCD types, processor variants, and feature sets, supporting easy adaptation and future expansion.

Scope Definition

This project addresses these challenges by developing a fully featured software-based digital clock system that:

- Eliminates external RTC dependencies
- Demonstrates fundamental embedded programming concepts
- Provides practical accuracy for non-critical applications
- Implements comprehensive user interface design patterns
- Serves as an educational platform for teaching embedded systems

Target Applications

- **Educational:** Embedded systems coursework and laboratory exercises
- **Hobbyist:** DIY electronics projects and maker communities
- **Prototyping:** Proof-of-concept for larger automation systems
- **General Purpose:** Home/office desk clocks, appliance displays

1.3 Objectives

Primary Objectives

1. Develop Accurate Software-Based Timekeeping
    - Implement interrupt-driven time management with ≤10ms resolution
    - Achieve timekeeping accuracy within ±5 seconds per 24-hour period
    - Design cascading counter system for hours/minutes/seconds tracking
    - Create robust millisecond accumulator with overflow handling
2. Implement Comprehensive Date Management
    - Design calendar arithmetic supporting dates from January 1, 2000 to December 31, 2099
    - Implement accurate leap year detection algorithm per Gregorian calendar rules
    - Create month-specific day validation (28/29/30/31 days)
    - Develop automatic date advancement logic at midnight transitions
3. Create Intuitive User Interface
    - Design multi-mode display system with time-primary and date-primary views
    - Implement keypad-based input system with proper debouncing
    - Develop visual feedback system using tri-color LED indicators
    - Create audible confirmation system using PWM-driven buzzer
    - Support both 12-hour (AM/PM) and 24-hour time format preferences

4. Ensure System Reliability
    - Implement comprehensive input validation for all user entries
    - Design error detection and reporting mechanisms
    - Create graceful error recovery procedures
    - Develop system reset functionality with user confirmation
5. Optimize Resource Utilization
    - Minimize SRAM usage to <5% of available memory (≤100 bytes of 2KB)
    - Limit Flash memory consumption to <25% of available space (≤8KB of 32KB)
    - Maintain CPU utilization below 10% average during normal operation
    - Optimize ISR execution time to <100μs per interruption [3]

Secondary Objectives

6. Provide Dual Implementation
    - Create functionally equivalent C language implementation
    - Develop parallel AVR Assembly language implementation
    - Document comparative analysis of both approaches
    - Demonstrate language-agnostic algorithm design
7. Maximize Extensibility
    - Design modular code architecture for easy feature additions
    - Create well-documented APIs for peripheral interfaces
    - Implement clear separation between hardware abstraction and business logic
8. Support Educational Use
    - Generate comprehensive technical documentation
    - Include detailed code comments explaining algorithms
    - Provide theoretical background for time/date calculations
    - Create testing procedures and validation guidelines

Chapter 2: System Architecture

Hardware Components

Table 2. 1 User Controls

Component	Port Assignment	Pin Configuration
LCD Display (16x2)	PORTB (Data), PORTD (Control)	Data: PB0–PB7 RS: PD0RW: PD1EN: PD2
4×4 Matrix Keypad	PORTC	Rows: PC0–PC3 Columns: PC4–PC7
LED Indicators	PORTD	Green: PD3 Red: PD4 Yellow: PD5
Buzzer	PORTD	PD6

System Block Diagram

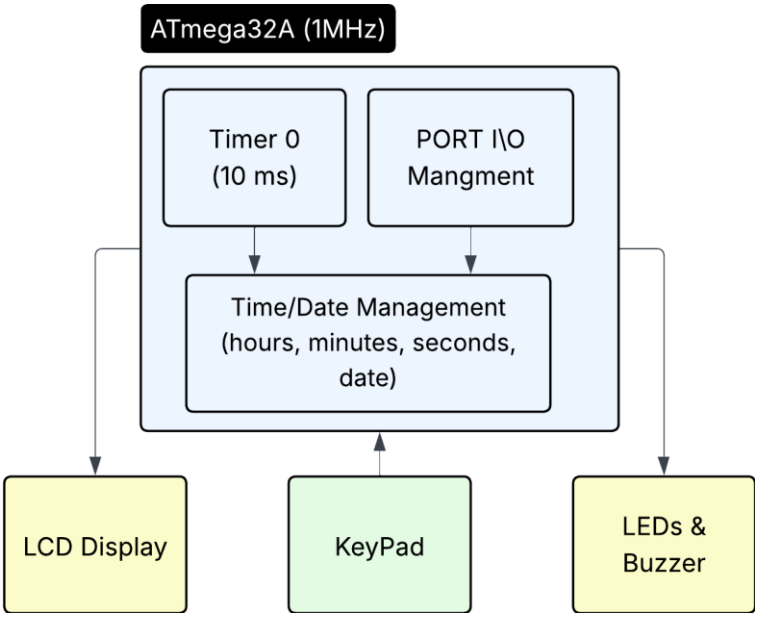

Figure 2. 1 Overall Block Diagram

Functional Specifications

Core Features

Time Management

- **12-Hour Mode:** Displays time with AM/PM indicator
- **24-Hour Mode:** Military time format
- **Precision:** ±1 second accuracy with software-based timekeeping
- **Range:** 00:00:00 to 23:59:59

Date Management

- **Format:** DD-MM-YYYY (numeric) or DD-MMM-YYYY (text)
- Year Range: 2000-2099
- **Leap Year Support:** Automatic leap year detection and February 29th handling
- **Month Validation:** Ensures valid day ranges for each month

User Interface

- Display Modes:
  - Mode 0: Time (large) with date (numeric)
  - Mode 1: Date (large with month name) with time (compact)
- **Input Method:** 4x4 matrix keypad with debouncing
- **Visual Feedback:** Three LED indicators for status
- **Audible Feedback:** Buzzer for key presses and alerts

User Controls

Table 2. 2 User Controls

Key	Function	Description
A	Set Time	Enter time setting mode with AM/PM selection
B	Set Date	Enter date setting mode with validation
C	Toggle Format	Switch between 12-hour and 24-hour modes
D	Toggle Display	Switch between time-primary and date-primary views
#	System Reset	Reset all settings to defaults (with confirmation)
*** **	Backspace	Delete last digit during input
0–9	Numeric Input	Enter digits for time/date values

Chapter 3: Software Design

Timer Configuration

Timer0 - CTC Mode (Clear Timer on Compare)
Configuration:

- Mode: CTC (WGM01 = 1)
- Prescaler: 64
- Compare Value (OCR0): 155
- Interrupt Frequency: 100 Hz (every 10ms) [3]

Calculation:
$\boldsymbol{Interrupt\ Period = (Prescaler \times OCR0) / F\_CPU = (64 \times 155) / 1,000,000}$ = 9.92 ms ≈ 10 ms

Keypad Scanning Algorithm

Matrix Scanning Technique:

1. Set all rows HIGH (idle state)
2. For each row (1-4):
    - Set current row LOW
    - Delay 3ms for signal stabilization
    - Read all columns
    - If any column is LOW: key detected at (row, column)
3. Return to idle state (all rows HIGH)

Debouncing Strategy:

- Initial detection delay: 10 ms with Re-scan after delay
- Release detection: Wait for key release before accepting next input
- Additional 20ms delay after key release

LCD Communication Protocol

8-bit Parallel Interface:
Command Sequence:

- Set data on PORTB
- Set RS (0=command, 1=data)
- Set RW = 0 (write)
- EN HIGH → delay 40µs → EN LOW
- Delay 2ms for command execution [4]

Initialization Sequence:

1. Power-on delay: 20ms
2. Function Set: 0x38 (8-bit, 2-line, 5x7 font)
3. Display Control: 0x0C (display ON, cursor OFF, blink OFF)
4. Entry Mode: 0x06 (auto-increment, no display shift)
5. Clear Display: 0x01
6. Return Home: 0x80

State Machine Design

Main Operation States

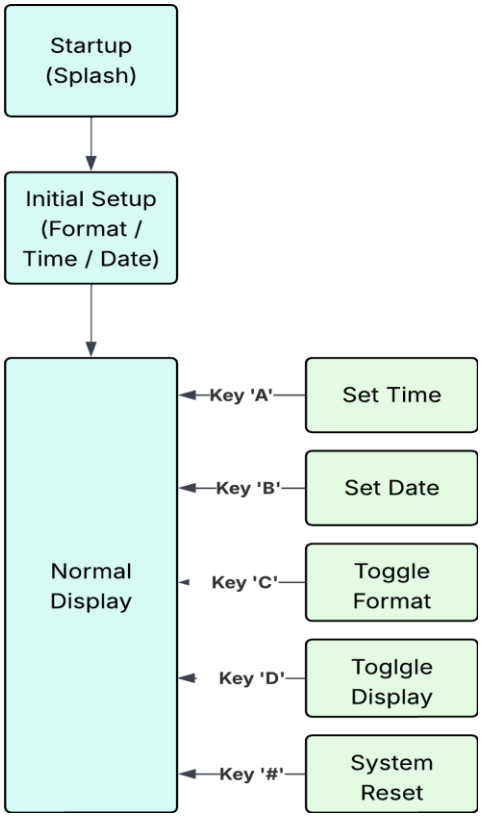

Figure 3. 1 State Machine Diagram

Input State Machine

Each input function (time/date setting) follows this pattern:

$$PROMPT \rightarrow INPUT\_DIGITS \rightarrow VALIDATE \rightarrow CONFIRM/ERROR \rightarrow RETURN$$

Chapter 4: Memory Management

SRAM Variables (60 bytes starting at 0x0060)

Table 4. 1 SRAM Variables

Variable	Size	Purpose
hours	1 byte	Current hour (0–23)
minutes	1 byte	Current minute (0–59)
seconds	1 byte	Current second (0–59)
milliseconds	2 bytes	Sub-second counter (0–999)
day	1 byte	Current day (1–31)
month	1 byte	Current month (1–12)
year	2 bytes	Current year (2000–2099)
mode_12hr	1 byte	Format flag (0 = 24 hr, 1 = 12 hr)
display mode	1 byte	View flag (0 = time, 1 = date)
line buffer	17 bytes	LCD string buffer (16 chars + terminator)
input buffer	10 bytes	User input buffer
temp variables	~15 bytes	Working registers / temporary variables

Memory calculation (digit-by-digit):

$$\mathbf{1} + \mathbf{1} + \mathbf{1} + \mathbf{2} + \mathbf{1} + \mathbf{1} + \mathbf{2} + \mathbf{1} + \mathbf{1} + \mathbf{17} + \mathbf{10} + \mathbf{15} = \mathbf{53} \text{ bytes}$$

Result: total SRAM usage ≈ 53 bytes — comfortably under your 100 bytes limit, leaving 47 bytes free for future use or stack/interrupt overhead (100 − 53 = 47).

Program Memory (Flash)

- C Implementation: ~4-5 KB
- Assembly Implementation: ~3-4 KB

String Constants: Stored in program memory with .db directives (Assembly) or PROGMEM (C) [2]

Chapter 5: Visual Feedback System

LED Indicators

Table 5. 1 LED Indicators

Pin	Color	Behavior	Meaning
PD3	Green	Blinks (one full on/off cycle per 1 s)	Heartbeat — system running
PD4	Red	ON while in date display mode	Display mode indicator
PD5	Yellow	ON for 3 seconds at top of hour	Hour marker

Buzzer Patterns

Table 5. 2 Buzzer Patterns

Pattern	Duration	Frequency	Usage
Short Beep	40 ms	~1 kHz	Key press confirmation
Long Beep	200 ms	~1 kHz	Important action (reset)
Flash Pattern	5 × 150 ms	1 kHz pulses	Warning (before reset)

Tone Generation (1 MHz clock):

$$\boldsymbol{Toggle\ period\ =\ 500\mu s\ (HIGH)\ +\ 500\mu s\ (LOW)\ =\ 1ms\ Frequency\ =\ 1\ /\ 1ms\ =\ 1000\ Hz}$$

Chapter 6: Input Validation and Error Handling

Time Input Validation

- Hours: 1–12 (12-hour mode) or 0–23 (24-hour mode)
- Minutes: 0–59
- Seconds: 0–59
- Automatic Clamping: Values exceeding valid limits are automatically adjusted to the nearest valid range.

12-Hour → 24-Hour Conversion:

- 12 AM → 00: xx: xx
- 01–11 AM → 01–11: xx: xx
- 12 PM → 12: xx: xx
- 01–11 PM → 13–23: xx: xx

Date Input Validation

1. Month Check: Must be between 1 and 12. If invalid, display “INVALID MONTH!”
2. Day Range Check:
    - January, March, May, July, August, October, December → 1–31
    - April, June, September, November → 1–30
    - February → 1–28 (or 1–29 if leap year)
3. Year Check: Must be 2000-2099 If invalid, display “INVALID YEAR!”
4. Leap Year Detection:
    - Year divisible by 4 → Leap year.
    - If February 29 is set → Display “LEAP YEAR!”

Note: If any validation fails, show the corresponding message and return without saving the data.

Error Messages

- “INVALID MONTH!” – Month out of range
- “INVALID DATE!” – Day or year invalid
- “LEAP YEAR!” – Informational notice after setting Feb 29

Power-On Initialization Sequence

1. RESET (0.0s)
   - Initialize stack pointers
   - Configure I/O ports
   - Initialize system variables

2. HARDWARE SETUP (0.0–0.1s)
   - Initialize LCD (≈20 ms + command delay)
   - Configure Timer0
   - Enable global interrupts

3. SPLASH SCREEN (1.2s)
   - Display: “DIGITAL CLOCK / WITH DATE”
   - Turn ON yellow and green LEDs
   - Hold for 1200 ms

4. INITIAL SETUP WIZARD (User-Driven)
   - Select format (12-hour or 24-hour mode)
   - Enter time with AM/PM selection
   - Enter date with full validation

5. NORMAL OPERATION
   - Enter main loop for continuous display updates

System Reset Functionality

Reset Procedure

When the user initiates a system reset, confirmation is required to avoid accidental data loss.

Reset Process:

1. The user presses the # key to trigger the reset request.
2. The Display shows the next "RESET SYSTEM? / A=YES B=CANCEL"
3. All LEDs flash, and the buzzer sounds 5 times as a warning signal.
4. The system then waits for user confirmation.

If the user confirms by pressing 'A':

- A long buzzer beep is playing.
- The display shows "RESETTING...".
- All variables are cleared:
    - Time is reset to 00:00:00.
    - Date is reset to 00-00-0000.
    - Mode is reset to 12hr.
- The system then displays the default time mode.
- A message "RESET COMPLETE! / RESTARTING..." appears.
- All LEDs flash for 1200 $\boldsymbol{ms}$.
- The system automatically re-runs the INITIAL_SETUP wizard.
- Normal operation resumes once setup is complete.

If the user cancels by pressing 'B':

- The display shows "CANCELLED".
- The yellow LED flashes once.
- The system returns to its previous state without making any changes.

Chapter 7: Design Rationale

Key Design Decisions

1. Software-Based Timekeeping

- Rationale: No external RTC chip required, reduces cost
- Trade-off: Lower accuracy vs. dedicated RTC, requires continuous power

2. 10ms Timer Resolution

- Rationale: Balance between accuracy and ISR overhead
- Alternative considered: 1ms resolution (higher CPU load)

3. Matrix Keypad vs. Individual Buttons

- Rationale: 4x4 matrix uses 8 pins vs. 16 for individual buttons
- Trade-off: More complex scanning logic

4. Separate Display Modes

- Rationale: 16x2 LCD limited space; users can choose emphasis
- Benefit: Better readability for preferred information

5. Leap Year Support

- Rationale: Essential for accurate long-term date keeping
- Implementation: Standard algorithm with 2000-2099 range

Extensibility Considerations

Future Enhancement Possibilities:

1. Alarm Function: Use EEPROM to store alarm times
2. Battery Backup: Add external RTC with battery for power-loss protection
3. Stopwatch Mode: Utilize millisecond counter
4. Calendar Functions: Day of week calculation

Limitations

1. Year Range: Limited to 2000-2099 (century calculation omitted)
2. No Battery Backup: Time/date lost on power failure
3. Single Time Zone: No automatic DST or time zone support
4. Input Validation: Limited error recovery during input
5. Display Size: 16x2 LCD constrains information density
6. Timekeeping Accuracy: Software-based timekeeping drifts without external calibration

Comparative Analysis: C vs Assembly Implementation

Performance Comparison

Table 7. 1 C vs Assembly Implementation

Metric	C Implementation	Assembly Implementation	Winner
Code Size (Flash)	4–5 KB	3–4 KB	Assembly
Development Time	~8–12 hours	~20–30 hours	C
Maintainability	High	Moderate	C
Execution Speed	Good	Excellent	Assembly
ISR Overhead	~80 µs	~50 µs	Assembly
Portability	High	Low	C
Debugging Ease	Easy	Challenging	C
Code Readability	High	Low	C

Key Observations

Advantages of C Implementation:

1. Faster Development: Built-in functions, operators, and control structures
2. Better Abstraction: Functions hide hardware complexity
3. Easier Debugging: Variable names, symbolic debugging, better toolchain support
4. More Maintainable: Clearer intent, easier to modify
5. Portable: Can target different AVR chips with minimal changes

Advantages of Assembly Implementation:

1. Smaller Code: Direct control over every instruction
2. Faster Execution: No compiler overhead, optimized register usage
3. Predictable Timing: Exact cycle counts for critical sections
4. Full Control: Direct hardware manipulation, no hidden operations
5. Educational Value: Deep understanding of processor architecture

Recommendations

Use C when:

- Development time is critical
- Multiple developers will maintain the code
- Portability across platforms is needed
- Code complexity is high
- Debugging support is essential

Use Assembly when:

- Flash memory is extremely limited
- Timing precision is critical
- Maximum performance is required
- Educational/learning objectives are primary
- Code size must be absolutely minimized

Conclusion

This digital clock system demonstrates a complete embedded application combining accurate timekeeping, date management, user interface design, and feedback systems. The dual implementation in both C and Assembly provides flexibility for different development scenarios and educational purposes.

Key Achievements:

- Accurate software-based timekeeping with 10ms resolution
- Full date management with leap year support
- Intuitive user interface with visual and audio feedback
- Robust input validation and error handling
- Efficient resource utilization (<5% CPU, 60 bytes RAM)
- Modular, maintainable code structure

Recommended Next Steps:

1. Addition of battery-backed RTC module
2. Implementation of alarm functionality
3. Temperature sensor integration
4. Custom enclosure design

Appendices

Appendix A: Schematics

A.1 Complete System Schematic

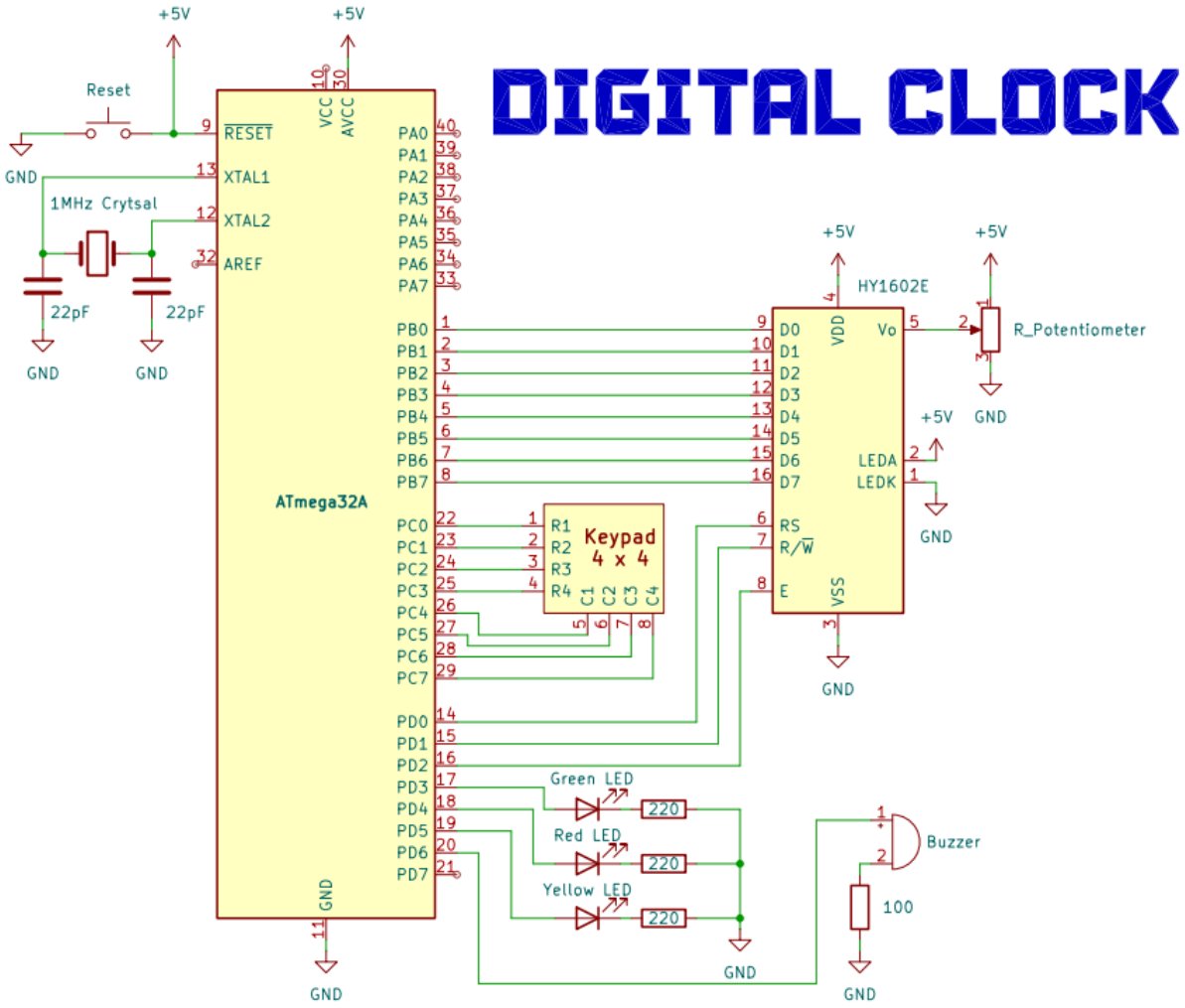

Figure A. 1 Complete System Schematic

A.2 Complete System Fritzing

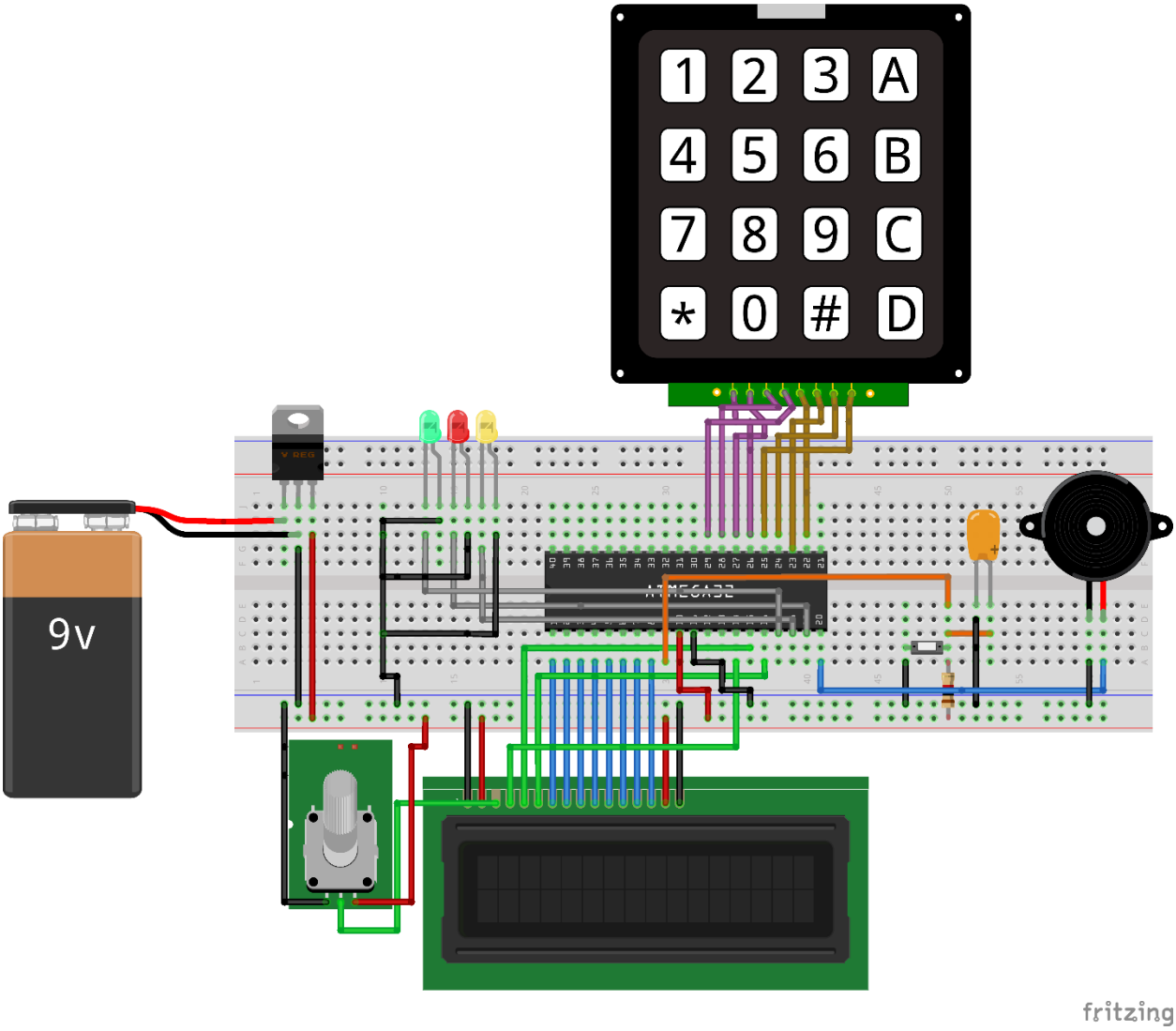

Figure A. 2 Complete System Fritzing

A.3 Complete System Architecture

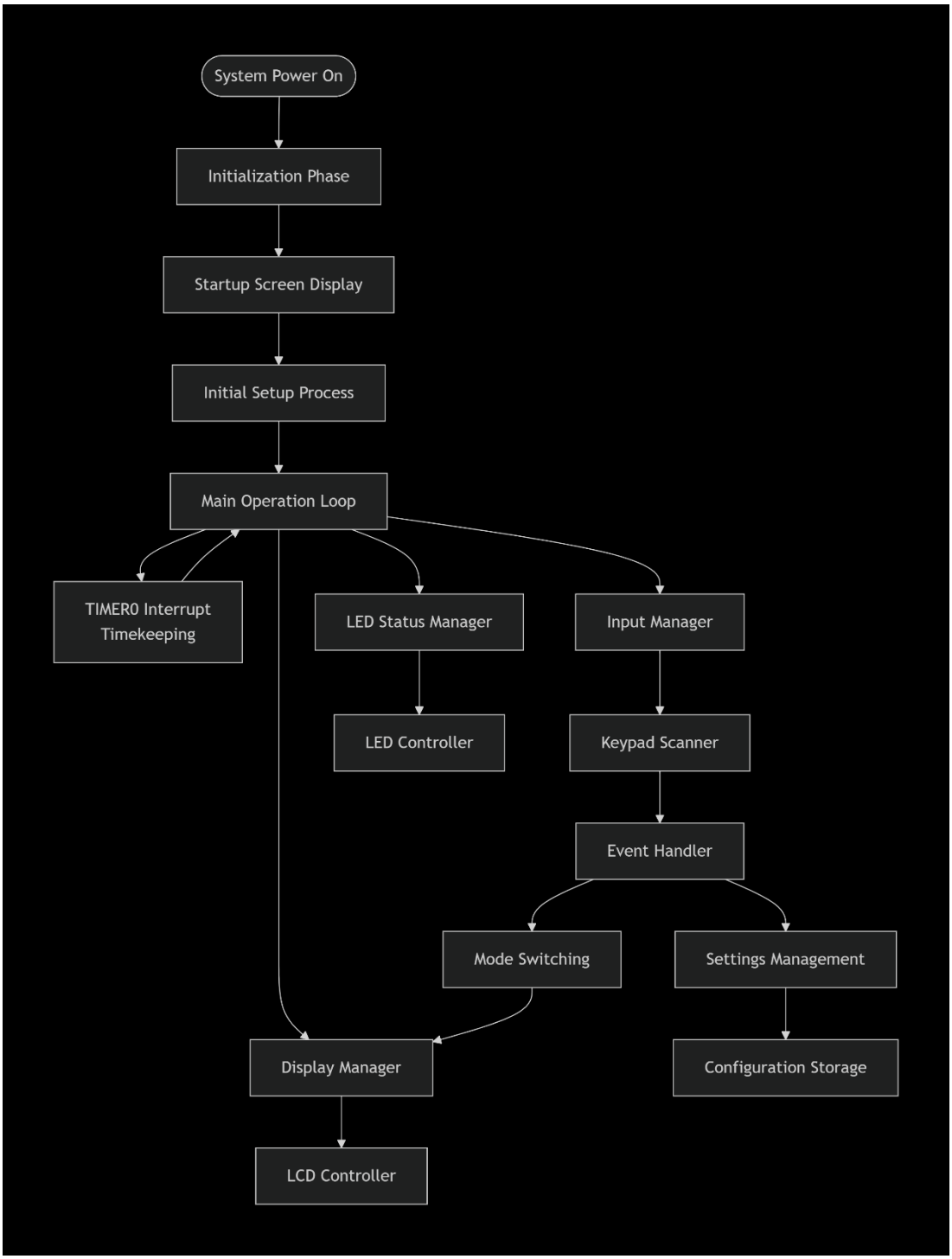

Figure A. 3 Complete System Architecture

A.3 LCD Connection Detail

Table A. 1 LCD Connection Detail

LCD Pin	Signal	ATmega32A Pin	Notes
1	VSS	GND	Ground
2	VDD	+5V	Power Supply
3	V0	Contrast Pot	10 $k\Omega$ potentiometer to GND
4	RS	PD0	Register Select
5	RW	PD1	Read/Write
6	EN	PD2	Enable
7–14	D0–D7	PB0–PB7	Data Bus (8-bit mode)
15	A	+5V	Backlight Anode
16	K	GND	Backlight Cathode

A.4 Keypad Matrix Connections

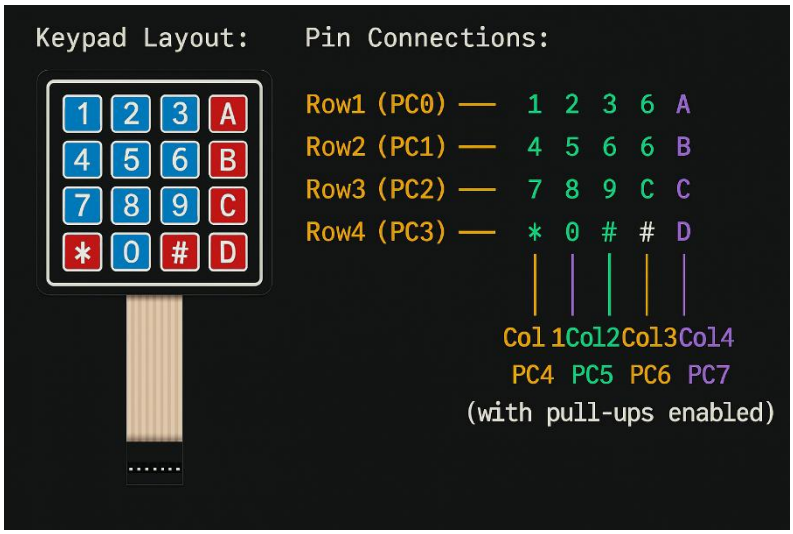

Figure A. 4 Keypad Matrix Connections

Appendix B: Timing Diagrams

B.1 LCD Enable Pulse Timing

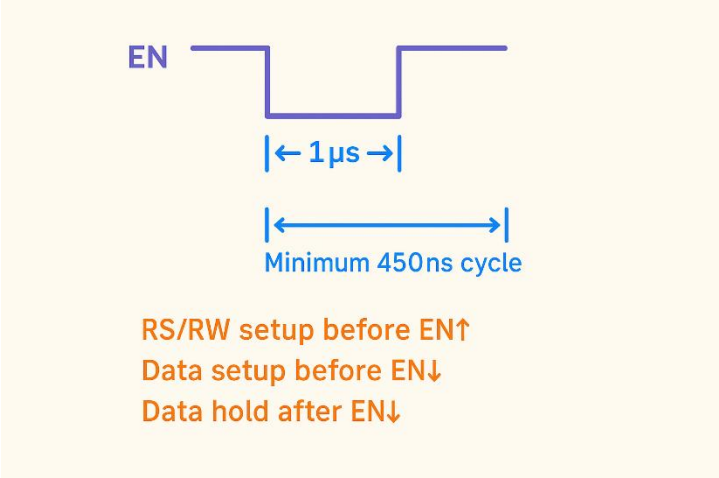

Figure B. 1 LCD Enable Pulse Timing

B.2 Timer0 Interrupt Timing (1MHz Clock)

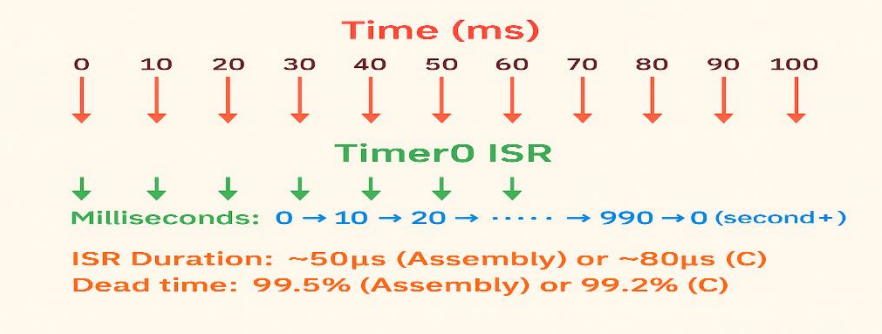

Figure B. 2 Timer0 Interrupt Timing (1MHz Clock)

B.3 Keypad Scan Timing

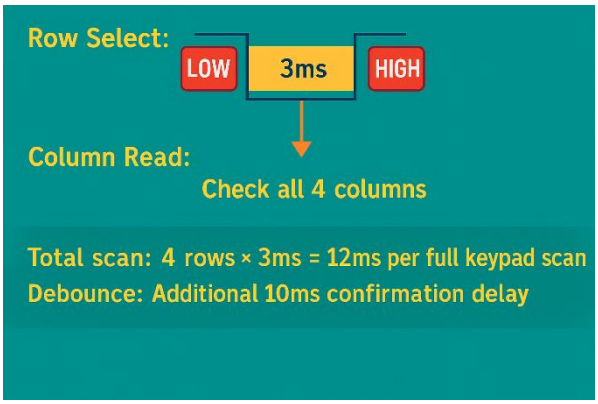

Figure B. 3 Keypad Scan Timing

Appendix C: Memory Maps

C.1 SRAM Memory Layout (ATmega32A)

Table C. 1 SRAM Memory Layout (ATmega32A)

Address Range	Content	Size	Purpose
0x0060–0x0060	hours	1 byte	Current hour (0–23)
0x0061–0x0061	minutes	1 byte	Current minute (0–59)
0x0062–0x0062	seconds	1 byte	Current second (0–59)
0x0063–0x0064	milliseconds	2 bytes	Sub-second counter (0–999)
0x0065–0x0065	day	1 byte	Day of month (1–31)
0x0066–0x0066	month	1 byte	Month (1–12)
0x0067–0x0068	year	2 bytes	Year (2000–2099)
0x0069–0x0069	mode_12hr	1 byte	Time format flag
0x006A–0x006A	display_mode	1 byte	Display mode flag
0x006B–0x006B	tick_flag	1 byte	ISR flag
0x006C–0x006C	second_flag	1 byte	Second change flag
0x006D–0x006D	last_second	1 byte	Previous second value
0x006E–0x006E	blink_state	1 byte	LED blink toggle
0x006F–0x007F	line_buffer	17 bytes	LCD string buffer
0x0080–0x0089	input_buffer	10 bytes	User input storage
0x008A–0x008F	temp variables	6 bytes	Working space
—	**Total Used**	**48 bytes**	**2.3% of 2 KB SRAM**

C.2 Flash Memory Layout

Table C. 2 Flash Memory Layout

Address Range	Content	Size (approx.)
0x0000–0x0001	Reset Vector	2 bytes
0x0002–0x0029	Interrupt Vectors	40 bytes
0x002A–0x0FFF	Program Code	3–5 KB
0x1000–0x1FFF	String Constants	500 bytes
0x2000–0x3FFF	Unused	~24 KB
—	Total Used	C: ~5 KB, Asm: ~4 KB → 12–16% of 32 KB

Appendix D: Bill of Materials and Cost Analysis (Egypt Market)

D.1 Complete Bill of Materials

Table D. 1 Complete Bill of Materials

Item	Specification	Qty	Unit (EGP)	Total (EGP)	Notes
ATmega32A	Microcontroller	1	290	290	RAM Electronics
LCD 16×2	HD44780	1	65	65	Common part
Keypad 4×4	Membrane type	1	35	35	Future Electronics
LED Green	5 mm, clear lens	10	0.5	5	—
LED Red	5 mm, clear lens	10	0.5	5	—
LED Yellow	5 mm, clear lens	10	0.5	5	—
Buzzer	5 V active, PCB mount	1	8	8	Passive alt. 5 EGP
Crystal	1 MHz, HC-49S	1	6	6	16 MHz alt. 5 EGP
Capacitor	22 pF ceramic	2	0.5	1	For crystal
Capacitor	100 nF ceramic	4	0.75	3	Decoupling
Resistor	330 Ω, ¼ W	3	0.25	0.75	LED limit
Resistor	10 kΩ, ¼ W	1	0.25	0.25	LCD contrast
Potentiometer	10 kΩ, PCB mount	1	3	3	Optional
IC Socket	40-pin DIP	1	6	6	Easy replace
Pin Header	Male 2.54 mm, 40-pin	1	4	4	For LCD/keypad
Jumper Wires	F-F 20 cm, ×40 pcs	1	25	25	Breadboard
Subtotal				**465.50 EGP**	

Note:
The **NTI Kit** was used for implementation instead of separate components, costing about **1500 EGP** and including all required parts.

APPENDIX E: PSEUDOCODE

E.1 General Concept of Code

```
START
  Init system (LCD, timer, ports)
  Setup: format, time, date
LOOP
  Timer ISR → update time/date
  Every 1s → refresh display, blink LEDs
  Keys: A=set time | B=set date | C=12/24h | D=toggle view | #=reset
  Auto date advance at midnight
REPEAT
DISPLAY
  Show time date or date time (mode)
LOGIC
  Validate inputs, handle leap years & month days
FEEDBACK
  LEDs & buzzer for status/events
RESET
  Confirm → clear → restart
```

Appendix F: Troubleshooting Guide

F.1 Common Issues

Table F. 1 Troubleshooting Guide

Problem	Likely Cause	Solution
LCD blank	Initialization failed	Check pin connections, verify delays
Keypad unresponsive	Port configuration error	Verify DDRC and PORTC settings
Time runs fast/slow	Timer calibration off	Adjust OCR0 value (±1–2)
Date doesn't advance	ADVANCE_DATE logic error	Check month/day validation
AM/PM incorrect	12-hour conversion error	Verify hour comparison logic
Buzzer silent	PWM frequency wrong	Adjust BUZZER_BEEP loop count

Appendix G: Assembly Mnemonics Reference

G.1 Commonly Used Instructions in This Project

Table G. 1 Assembly Mnemonics Reference

Instruction	Meaning	Description
ldi r24, value	Load Immediate	Load an 8-bit constant value into register r24
lds r16, addr	Load from SRAM	Load a byte from SRAM address addr into r16
sts addr, r16	Store to SRAM	Store the contents of r16 into SRAM address addr
in r16, port	Input from I/O Port	Read from I/O port into register r16
out port, r16	Output to I/O Port	Write the value in r16 to I/O port
cp r16, r17	Compare Registers	Compare values in r16 and r17 (affects flags)
brne label	Branch if Not Equal	Jump to label if the previous compare was not equal
reti	Return from Interrupt	Return from an interruption routine and re-enable interrupts
sei / cli	Set/Clear Interrupt Flag	Enable (sei) or disable (cli) global interrupts

Appendix H: Register Usage Convention

Register Allocations

Table H. 1 Register Allocations

Register(s)	Name / Alias	Description
r0	Temporary	Used as a temporary register (also by MUL)
r1	Multiply result high	Holds the high byte of the multiplication result
r16–r27	Working registers	General-purpose registers for computation
r28–r29	Y register	General-purpose or indirect addressing pointer
r30–r31	Z register	Pointer register used for indirect addressing
r26–r27	X register	Pointer registers for SRAM access

Appendix I: Compile and Build Instructions

Required Tools

- Atmel AVR-ASM (or AVRA)
- AVRdude (for programming)
- ATmega32A with ISP/JTAG interface

Build Commands

$$avra\ -o\ clock.hex\ digital\_clock.asm$$

$$avrdude\ -p\ m32\ -c\ <programmer>\ -U\ flash:w:clock.hex$$

Fuse Settings (1MHz internal RC)

- Low Fuse: 0xE4
- High Fuse: 0xD9
- Extended Fuse: 0xFF

Appendix J: Source Code

The complete implementation is available here:
https://github.com/Muhammad-296/Digital-Clock/tree/main

Appendix K: Software and Hardware Implmentation

K.1 Software implementation

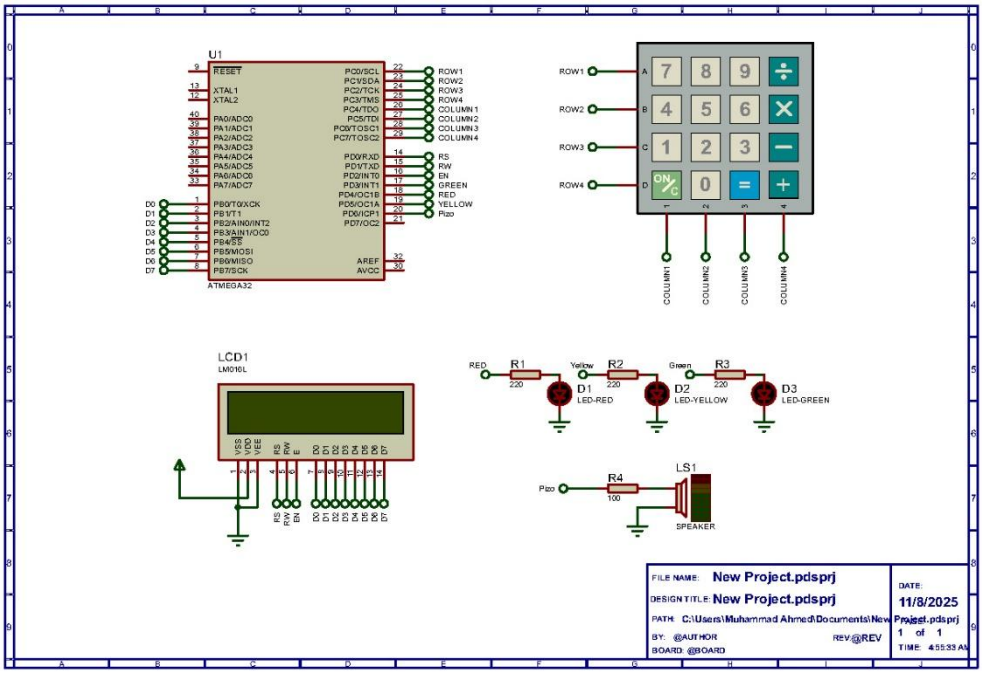

Figure K. 1 Software implementation

K.2 Hardware implementation

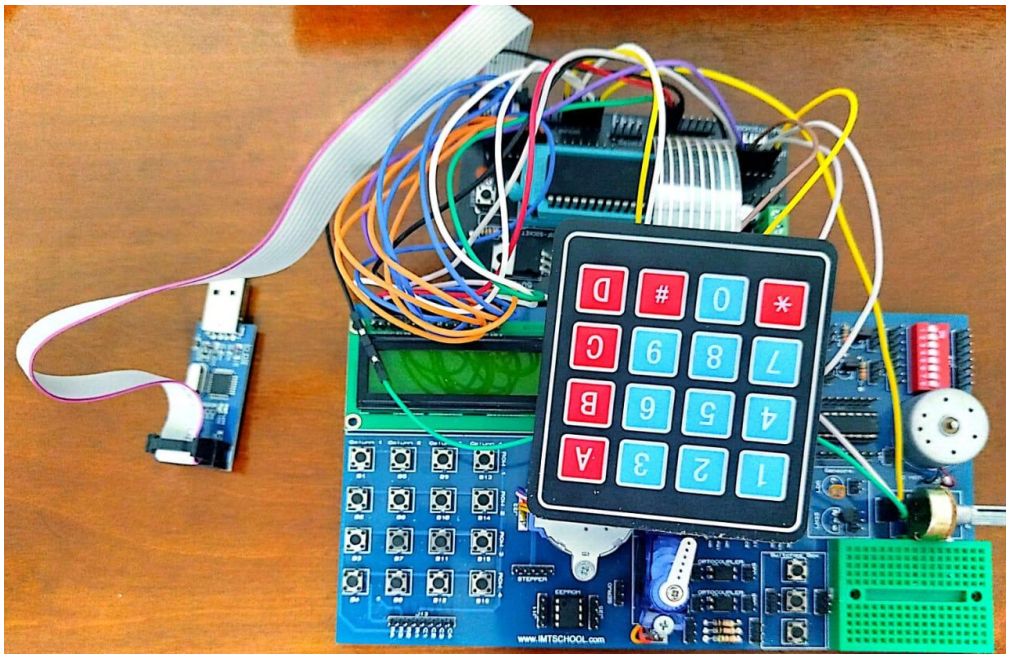

Figure K. 2 Hardware implementation

References

[1] J. W. Valvano, Embedded Systems: Real-Time Interfacing to the ARM Cortex-M Microcontrollers, 1st ed, 2011.

[2] M. T. Inc, "ATmega32A Datasheet," 2019. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/ATmega32A-DataSheet-Complete-DS-DS40002061A.pdf. [Accessed 2025].

[3] Atmel, "Interfacing 4×4 Matrix Keypad with AVR Microcontrollers," 2025. [Online]. Available: https://ww1.microchip.com/downloads/en/AppNotes/doc8362.pdf. [Accessed 08 Novamber 2025].

[4] Hitachi Ltd, "HD44780 LCD Controller Datasheet," 1999. [Online]. Available: https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf. [Accessed 08 Novamber 2025].